AI Interview Platform Product Engineer Case Study

Problem analysis, revamp strategy & fullstack execution

> The platform was converting **absence of evidence into evidence of incompetence** — a skill the interview never reached was written into a candidate's permanent record as L1, *"cannot apply independently"*, and the summary that should have revealed it counted four of seven skills. This report traces that defect from the database constraint to the rendered pixel, and the change that removes it.

Candidate	**Albert Ade Kristadeo**
Branch	**`feat/fit-gap-integrity`**
Scope	**api/ · Rails 7 | web/ · React 18 + TypeScript**
Tests	**54 RSpec · 18 Vitest — both built from zero**

Candidate: Albert Ade Kristadeo **Scope:** `api/` (Rails 7 · PostgreSQL · Sidekiq) and `web/` (React 18 · TypeScript · Vite · Tailwind · Radix)

Submission links

Video walkthrough	https://youtu.be/bs6h74YCspc
Pull request	https://github.com/rakamindev/ai-interview-platform/compare/main...myst-al:feat/fit-gap-integrity
Branch	`myst-al/ai-interview-platform` → `feat/fit-gap-integrity`
Seeded fault branch	`spike/seeded-fault` — fault `075e96e`, revert `49f1636`

The one-paragraph version

The platform runs an AI interview, grades a candidate against behavioural anchors, and hands a recruiter a fit/gap report. I found that the report could not be trusted, for a reason that runs from the database to the browser: **the schema had no way to record "we did not assess this."** `portfolio_skills.ai_level` was `NOT NULL CHECK (1..5)`, so the generator pushed every missing rating through `.to_i.clamp(1, 5)` — turning `nil` into **1**, the anchor that reads *"cannot apply independently."* The frontend independently applied the same fallback. The fit/gap summary then counted only match, exceed and gap, so a seven-skill role reported on four and read as complete. A skill the interview ran out of time to reach became a documented weakness on a real person's evaluation, and nothing on screen revealed the arithmetic. I made absence representable, made every layer carry it honestly, and built the test harness that proves it.

Alongside that I found and fixed a **cross-tenant data exposure**: `Portfolio` and everything hanging off it carry no `tenant_id` and were fetched with bare `.find(params[:id])`, so any authenticated assessor could read — and, through the override endpoint, **write to** — another organisation's candidate.

Step 1 — Setup & Local Exploration

What I found

The repository documents a host-native setup: Ruby 3.3.2, PostgreSQL, Redis. I had Docker and Node but none of the three. Rather than install a Ruby toolchain I did not want to maintain, I containerised the backend — which also made the test suite reproducible, and would make CI trivial later.

The shipped `api/Dockerfile` cannot run tests. It builds with:

```
RUN bundle config set --local without 'development test' && \
    bundle install --deployment --frozen --jobs 4
```

That is correct for production and fatal for RSpec — `rspec-rails` and `factory_bot_rails` live in exactly those groups. So I added `api/Dockerfile.dev` installing every group, plus a `docker-compose.yml` covering api, sidekiq, postgres and redis.

Friction worth recording

Three things cost time that should not cost anyone time again. All three are fixed in the PR.

Problem	Effect	Fix
`web/.env.example` points at `localhost:3000`	The API serves on **3001**. A fresh clone following the docs cannot reach the backend at all — every request fails.	Corrected, with a comment naming the authority (`api/config/puma.rb`).
Windows checkout rewrote `api/bin/*` to CRLF	The Linux container reads the shebang as `ruby\r` ; every `rails` and `rspec` call prints `/usr/bin/env: 'ruby\r': No such file or directory` .	`.gitattributes` pinning `eol=lf` .
Gemini model ids are stale **and** self-contradictory	`application.yml.sample` says `gemini-2.0-*` ; `api/README.md` says `gemini-3.1-flash-live-preview` . Neither resolves for a newly issued key. The API 404s, the client raises, and the fit/gap engine falls back to its mechanical narrative — so it looks like a content problem, not a config one.	Documented the probe command in `.env.example` , including the trap that a model can be *listed* and still refused.

That last one is the interesting one, and I only understood it because of a change I had already made: I gave fit/gap reports a `narrative_status` column. Without it, a 404 from Google is indistinguishable from a report that simply reads a bit flat.

Reachability

`db/seeds.rb` creates the organisation and the 22-skill taxonomy — and nothing else. There is no assessment, session, transcript, portfolio or vacancy, so **the portfolio and fit/gap screens are unreachable without conducting a live AI interview end to end.** For an exercise that asks for screenshots of those screens, that is a wall.

I added `rails demo:seed` , which builds one fully assessed candidate chosen to land on every branch of the comparison — match, exceed, gap, both not-assessed reasons — plus an assessor override, a discovered skill, a medium-confidence rating and a deliberately 255-character skill label. All candidate text is synthetic. It is idempotent and has a `demo:reset` companion.

That task is the reason the rest of this report has evidence in it rather than assertions.

Step 2 — Deep Context & Domain Immersion

The product

An assessor configures an assessment: a set of skills, each with five behavioural anchors (L1–L5) drawn from a shared taxonomy, and a time limit. A candidate joins by invite link and is interviewed by Gemini over a live audio session. A coverage map tracks, per skill, whether it has been probed and how deeply. When the session ends, a background job reads the full transcript and the coverage map and produces a **portfolio**: a level, a confidence, two or three verbatim quotes as evidence, and a competency summary per skill. An assessor may override any level. Finally the portfolio is compared against a **vacancy** — a role with its own expected levels — producing the **fit/gap report** that a hiring manager actually reads.

The chain matters: *transcript → coverage → portfolio → override → fit/gap → PDF*. Each step compresses. By the last one, forty minutes of a person's speech has become a row in a table that says "Gap −2". Everything I found is about what gets lost in that compression and whether the loss is visible.

The industry

Indonesian white-collar hiring runs at a volume that makes per-candidate human attention economically impossible at the top of the funnel. A posting for a mid-level engineering role in Jakarta routinely draws hundreds of applicants. What is already commoditised is *filtering*: CV parsing, keyword matching, coding challenges, personality inventories. Those are cheap and they are all proxies — they measure artefacts of a career rather than the thing you want to know.

The leverage in this product is not that it filters faster. It is that a structured interview against explicit behavioural anchors is **a better measurement instrument than a CV**, and it can be given to everyone rather than only to the shortlist. That is a genuine equity argument: candidates from non-target universities, career changers, and people without the vocabulary of the incumbent industry are precisely the ones a CV screen discards and a behavioural interview can rescue.

That argument only holds if the instrument is honest about its own uncertainty. An assessment tool that reports a confident level where it has no evidence is worse than a CV screen, because it launders a guess into a number and puts an institutional stamp on it. Which is what I found it doing.

What it is for

The outcome is **a defensible hiring decision made faster than a human panel could make it, without being worse.**

For that to keep being true, three things must hold:

1. **The levels must mean what the anchors say they mean.** L2 must be L2 because the transcript shows L2 behaviour.
2. **The report must distinguish evidence from absence of evidence.** These are different findings and must never be rendered the same way.

3. **A human must be able to disagree, and their disagreement must survive.** The assessor override is the system's only correction mechanism.

I found all three broken. (1) by the L1 coercion, (2) by the missing `not_assessed` accounting, (3) by regeneration silently destroying overrides.

The users

Assessors design assessments and review portfolios. They are the only people in the loop with both the domain knowledge to judge a level and the mandate to change it. Their scarcest resource is attention: they will review the flagged, not the routine. Anything the UI does not flag, they will not look at.

Recruiters and hiring managers consume the fit/gap report, often without opening the portfolio behind it. For them the report *is* the candidate. They are reading it between other meetings, at a glance, looking for a reason to advance or decline. A summary line that says "1 match, 1 exceeds, 2 gaps" is, for this reader, the whole truth — they will not count rows to check whether it covers the whole role.

That reading behaviour is why the missing `not_assessed` count is a P0 and not a cosmetic omission. The defect is calibrated precisely to the way the report is actually read.

The people affected who never chose it

Candidates. They do not select this tool, cannot opt out of it without withdrawing from the role, and receive no copy of what it concluded about them. A wrong level changes their year.

Three consequences shaped what I built:

Absence must never read as deficiency. If the interview hits its time ceiling before reaching Security Engineering, the candidate has not demonstrated weakness in security — nothing happened. Recording that as L1 is not a rounding error; it is the system inventing an adverse finding. This is the single most important thing I fixed, and I fixed it at the schema level so no future code path can reintroduce it.

Uncertainty must not be laundered into confidence. The discovered-skills list rendered `medium` confidence with the word **"confirmed"**. A model's medium-confidence inference, shown to a hiring manager as confirmed, is an overstatement that the candidate has no opportunity to contest.

Provenance must be visible. When an assessor overrides a level, the report showed the new number with no indication a human had touched it — the `is_override` flag was computed and then dropped before serialisation, while the UI legend advertised a marker that could therefore never appear. A reviewer could not tell model judgement from human judgement, in either direction.

UU PDP

Law No. 27 of 2022 on Personal Data Protection is directly engaged here. Interview transcripts, verbatim quotes and competency evaluations are personal data; some of it — spoken answers about work history, education, and by inference health or family circumstances — is sensitive. The obligations that bit in this codebase:

Principle	What I found
Purpose limitation & access control	`Portfolio.find(params[:id])` with no tenant scope. An assessor at Org A could read Org B's candidate — including the PDF export with verbatim quotes — by incrementing an integer. The override endpoint allowed cross-tenant **writes**. This is the most serious finding in the codebase.
Data minimisation in logs	`generation_error` stored `e.message` verbatim and logged it. A `JSON::ParserError` message embeds the fragment it failed to parse — which is the model's rendering of what the candidate said. Candidate speech into application logs, and into a DB column served back over the API.
Accuracy	A processor must not hold inaccurate personal data. Recording an unmeasured skill as L1 is the storage of a factual claim about a person that the system has no basis for.
Rights of the data subject	Not addressed by this codebase and out of scope for this change — see Constraint Signal.

Step 3 — Problem & Gap Analysis

Method

I did not read the code first. I ran the workflow, seeded a candidate designed to hit every branch, generated a real fit/gap report, read the persisted JSONB, then opened the browser and compared what the database held against what the screen said. Every finding below was reproduced against running software before I touched it; the ones I could not reproduce are recorded as dropped.

Severity

P0 — produces a materially wrong evaluation, or exposes personal data. **P1** — the workflow fails or misleads, recoverable by a knowledgeable user. **P2** — visible defect, no wrong decision. **P3** — hygiene.

Findings

P0-1 · An unassessed skill is recorded as L1

`api/app/services/portfolios/generator.rb`

```
ai_level: skill_data['level'].to_i.clamp(1, 5)
```

`nil.to_i` → `0` → `.clamp(1,5)` → `1`. Same for `""`, `"three"`, and a genuine `"L4"` string. The schema permitted nothing else: `ai_level` was `NOT NULL` with `CHECK (ai_level >= 1 AND ai_level <= 5)`.

> **Impact:** A skill the interview never reached is written into the candidate's permanent record as *"Executes with explicit guidance and close review. Cannot apply independently."* The recruiter sees a documented weakness where no measurement occurred.

Note this is a **missing specification**, not a bug: `not_assessed` exists in the `fit_result` enum, so somebody thought about this case at the report layer and nobody carried it back to storage.

P0-2 · The frontend applies the same coercion independently

`web/src/utils/constants.ts`

```
export function parseLevel(level: string | number): number {
  ...
  return isNaN(n) ? 1 : n;   // ← same defect, other end of the stack
}
```

> **Impact:** Fixing the API alone would not have fixed the product. Two independent layers had each decided that "unknown" means "lowest".

P0-3 · The Required column is blank on every row

`web/src/types/index.ts` ↔ *`api/app/services/fit_gap/engine.rb`*

The client declared `required_level: number` as a **required** field. The API emits `expected_level`. `required_level` appears **zero times** in the entire `api/` directory — I grepped to confirm before believing it.

`LEVEL_LABELS[undefined]` renders nothing, so every row of the comparison table showed a blank Required cell. The same data renders correctly in the PDF export, which reads `expected_level` — so the identical report is right on paper and broken in the browser.

> **Impact:** A gap report that cannot show what the gap is measured *from*. "⚠ Gap −2" with no target beside it.

TypeScript could not catch this: the payload crosses an `axios` boundary with an unvalidated cast, so the declared type was never checked against the wire.

P0-4 · The summary omits unassessed skills entirely

`ComparisonTable.tsx` *and* *`FitGap::Engine#generate_fallback_narrative`*

Both counted match, exceed and gap. Neither counted `not_assessed`. Reproduced against the seeded candidate — seven required skills, three unassessed:

```
✅ Match: 1   ⚠ Gap: 2   ⭐ Exceeds: 1
```

Four of seven. The other three are in no count, no total, and no sentence.

> **Impact:** The reader is given a number that looks like a complete evaluation of the role and is not. This is the finding I consider most damaging, because it is invisible by construction — nothing on screen prompts you to check.

P0-5 · Cross-tenant read and write of candidate personal data

`api/app/controllers/api/v1/portfolios_controller.rb`, *`portfolio_skills_controller.rb`*

`TenantScoped` is included by `Assessment`, `Session` and `Vacancy`. `Portfolio` has no `tenant_id` column and no scoping. Every lookup was:

```
portfolio = Portfolio.find(params[:id])          # global, unscoped
```

The override endpoint looked defended and was not:

```
PortfolioSkill.joins(:portfolio).find(params[:id])
```

`joins(:portfolio)` adds no tenant predicate when `Portfolio` itself is unscoped.

> **Impact:** Any authenticated assessor could read another organisation's candidate portfolio and download the PDF with verbatim interview quotes, by incrementing a sequential integer. Through the override endpoint they could **alter another organisation's hiring evaluation**, recorded against their own user id. UU PDP purpose-limitation breach.

P0-6 · A malformed model response destroys the portfolio

`Portfolios::Generator#save_skills`

```
portfolio.portfolio_skills.destroy_all      # ← no transaction
(data['configured_skills'] || []).each do |s|
  portfolio.portfolio_skills.create!(...)   # ← can raise here
end
```

`destroy_all` ran first, outside a transaction, before anything was validated. A response that failed validation midway left the portfolio **empty** and marked `failed`. `PortfolioGeneratorWorker` has `retry: 3`, so a consistently malformed response destroyed it four times.

Worse: `has_one :assessor_override, dependent: :destroy` means `destroy_all` **cascades into the assessor overrides** — so every regeneration silently discarded the only human judgement in the system, with no way to recover it.

> **Impact:** Irreversible loss of both the AI assessment and the assessor's corrections, triggered by a bad response from a third party.

P0-7 · Candidate speech persisted into error fields and logs

`Portfolios::Generator`, *`PortfolioGeneratorWorker`*

```
portfolio.update!(generation_status: 'failed', generation_error: e.message)
Rails.logger.error("... #{e.class} #{e.message}")
```

A `JSON::ParserError` message contains the fragment of the document it choked on. That document is the model's summary of the candidate's interview.

> **Impact:** Candidate personal data written to application logs and to a database column that `PortfoliosController#show` returns to any assessor. Retained indefinitely, outside the purpose it was collected for.

P1-1 · Override provenance discarded before serialisation

`FitGap::Engine#effective_portfolio_skills` computed `overridden: override.present?` and the comparison hash built four lines later did not include it. The UI legend read "✒ = **human override applied**" — a marker that could never render, because the flag never arrived.

> **Impact:** A reviewer cannot distinguish a model rating from a human one. The legend actively asserts a capability that does not exist.

P1-2 · Medium confidence displayed as "confirmed"

```
{s.ai_confidence?.toLowerCase() === "low" ? "low confidence" : "confirmed"}
```

`medium` fell into the else branch.

> **Impact:** A medium-confidence inference presented to a hiring manager as confirmed fact.

P1-3 · The failure narrative is mislabelled as a culture assessment

When the model call fails, `culture_narrative` is `nil` and the UI falls back to `overall_narrative` — the mechanical skill count — under the heading "**Culture & Competency Fit**", with nothing marking it as degraded.

> **Impact:** A sentence counting skills is read as a judgement about how a person works.

P1-4 · Polling never terminates

`usePolling(fetchReport, 5000, generating && ...)` — `generating` is only cleared by success. When the job exhausts its Sidekiq retries, the spinner turns forever.

> **Impact:** No terminal state, no error, no retry affordance. The assessor waits on something that is never coming.

P1-5 · Every non-404 error is swallowed

`fetchReport`'s `catch` handles 404 and silently discards everything else, leaving `loading: false`, `report: null` — a blank page with a header.

P1-6 · Duplicate vacancy labels silently drop a requirement

`vacancy_skills.index_by(&:skill_label)` keeps only the last. Nothing in the schema forbids two skills sharing a label.

> **Impact:** A stated requirement vanishes from the report with no error.

P1-7 · An empty vacancy crashes the worker

`validates :skill_comparisons, presence: true` — and `[]` is blank in Rails. A vacancy with no skills raised `RecordInvalid` inside the job, which then retried and never succeeded.

P2-1 · `web/.env.example` points at the wrong port

P2-2 · `PortfolioSkill` TypeScript type misdescribes the wire

`ai_level: string; // "L1" | "L2"...` — the API sends the integer `4`. `skill_id?: number` — the API sends `"SK-ENG-001"`. The discovered-skills list printed a bare `3` because its author trusted the annotation. **The wrong type caused the rendering bug.**

P2-3 · `LevelRadio` emits duplicate DOM ids

`id={` level-${level} `}` — one radio group per skill card, so `level-1` … `level-5` repeat N times per page. Clicking a label can activate a different skill's radio; assistive tech cannot resolve the reference.

P3-1 · Stale/contradictory Gemini model ids across sample config and README

P3-2 · CRLF line endings break container shebangs

P3-3 · `db/seeds.rb` interpolates into raw SQL

Constants only, so not exploitable — but the pattern invites a future injection.

Missing specification vs defective implementation

The brief asks for this split, and it turned out to be the most useful lens I applied.

Never specified. Nobody decided what an unassessed skill *is* (P0-1). The `fit_result` enum has `not_assessed`, so the question was answered at the report layer and never carried back to storage. Nobody decided whether an override survives regeneration (P0-6), or what a fit/gap report should say about its own degradation (P1-3). These are product decisions that were never made, and the code filled the vacuum with defaults that happened to harm candidates.

Specified and broken. The frontend was written against a contract (`required_level` , `is_override`) that the backend does not honour (P0-3, P1-1). The override legend, the `overridden` computation, and the `not_assessed` enum all show the intent existed. `joins(:portfolio)` (P0-5) is defensive code that defends nothing. These are not missing features; they are features that were built and then silently disconnected.

The distinction changes the fix. Defective implementations get repaired. Missing specifications needed a decision from me first — which is what Step 4 records.

Constraint signal — what I would escalate

Things I would raise with a Technical Lead before this ships, which I have deliberately **not** resolved unilaterally:

1. **Tenancy has no test that fails when someone adds a model.** I fixed the four models I found. The next unscoped model reintroduces the hole. This needs an architectural guard — a lint, a shared concern all session-descended models must include, or a request-level assertion — not four patches. **I would not call this closed.**
2. `users` **has no** `tenant_id`**.** JWTs carry a `scheme` claim resolved to an org by middleware, and `users` is a flat global table shared with `rakamin-api`. `assessor_overrides.overridden_by` is an unconstrained bigint with no FK. I could not determine from this repo alone whether user↔org membership is enforced upstream. **If it is not, my tenancy fix is necessary but not sufficient** — an attacker with a valid JWT for the wrong scheme is a different question. This is the single thing I would want answered before calling the security work done.
3. **UU PDP data-subject rights are unimplemented.** No retention policy on transcripts, no erasure path, no export for the candidate, no record of a lawful basis. Candidates receive no copy of conclusions drawn about them. Out of scope for one change; a compliance gap someone must own.
4. `portfolios.candidate_id` **and** `sessions.candidate_id` **are nullable bigints with no foreign key.** Portfolios can be orphaned from the person they describe.
5. **The prompt is the real business logic and is untestable.** Level assignment lives in a heredoc. I improved it — it now instructs the model to return `null` rather than guess — but "does this prompt produce calibrated levels?" cannot be answered by the test suite I built. That needs an evaluation harness with a labelled transcript set. It is the highest-value engineering investment this product could make and it is much larger than this exercise.
6. `retry: 3` **on a job that mutates before validating** was actively harmful. I made the write atomic, but retry semantics for AI-backed jobs deserve a deliberate policy.

Step 4 — Revamp Strategy, Acceptance Criteria & Trade-offs

Self-derived acceptance criteria

Written before implementation. Each is now an executing test.

Level coercion

Input	Required behaviour
`3`	`ai_level = 3`
`"L4"`, `"4"`	`ai_level = 4`
`nil`, `""`, `"three"`, `0`, `9`	`ai_level = nil`, never `1`
level absent	`ai_confidence` forced to `low`
`confidence: "very high"`	degrade to `low`, do not fail the batch

Partial writes

- An unusable response leaves existing skills untouched.
- A failure on row 2 of 3 leaves the portfolio exactly as it was.
- Assessor overrides survive regeneration, retaining the `ai_level` that was overruled.
- A response with only `discovered_skills` is valid.
- A row with no `skill_label` is skipped, not fatal.

Fit/gap

- No portfolio entry → `not_assessed` / `not_covered`.
- Entry present, `ai_level` nil → `not_assessed` / `insufficient_evidence`.
- An unrated skill is **never** `gap`, whatever the expected level.
- Overridden rows carry `is_override: true` and the model's original `ai_level`.
- Two vacancy skills sharing a label both appear.
- A vacancy with no skills produces an empty report, not an exception.
- The fallback narrative names every unassessed skill and states it is mechanical.

Presentation

- The Required cell is populated on every row, including unassessed ones.
- Summary counts sum to the number of required skills.
- Empty, error, loading, timeout, long-text and mobile states all render.

- A 255-character label neither overflows nor is lost from the DOM.

Data safety

- No transcript content in `generation_error` or in any log line.
- Every migration is reversible, or refuses with an explanation.
- Cross-tenant access raises `RecordNotFound`.

Trade-offs

Decision 1 — How to represent "not assessed"

Option A — nullable `ai_level`. One migration, `nil` means unassessed. Minimal surface, but conflates "never asked" with "asked, learned nothing", and every consumer must remember to null-check.

Option B — an `evidence_state` enum column (`assessed` / `insufficient_evidence` / `not_covered`) alongside a nullable level. Self-documenting and queryable.

Option C — a sentinel level, e.g. `0`. No migration. Rejected outright: it puts an invalid value inside a validated range and every arithmetic path (`delta = candidate − expected`) silently produces nonsense. This is how the L1 bug happened in the first place.

Chosen: A, with the distinction derived at the report layer. `ai_level` becomes nullable; `FitGap::Engine` computes `not_assessed_reason` from whether a portfolio row exists at all. This gets Option B's expressiveness in the report — which is where a human reads it — for one nullable column instead of a new enum plus backfill plus a write path through the generator.

What it forecloses: you cannot query "all insufficient-evidence skills" directly in SQL. If that becomes a product need — say, "which skills does our interview keep failing to reach?", which is a genuinely valuable question — Option B is the migration to write, and the derivation logic already defines its semantics. I judged that speculative for this change. Cheap to walk back.

Decision 2 — Which side of the `required_level` / `expected_level` break to fix

Option A — rename the API to `required_level`. Matches the client's existing declaration. But `skill_comparisons` is persisted **JSONB**: every historical report holds `expected_level` and would need a data migration to keep rendering. It also breaks `Exports::PdfGenerator`, which reads `expected_level` correctly today, and diverges from the `vacancy_skills.expected_level` column the value comes from.

Option B — fix the client to read `expected_level`.

Chosen: B. It repairs every already-persisted report for free with no backfill, keeps one name from database column through service to PDF, and touches two files. Option A is more code, more risk, and leaves the schema disagreeing with itself.

Failure mode: a client deployed before the API would still render blank. Given they release as one unit, that is not a live risk here — but on a split release it would need ordering.

Decision 3 — How to scope tenancy for `Portfolio`

Option A — add `tenant_id` to `portfolios` and use `TenantScoped`. Consistent with existing models. But it denormalises a value that already exists on `sessions`, needs a backfill, and creates a column that can drift out of sync with the session it was copied from.

Option B — a `default_scope` joining through `session`. Automatic, impossible to forget. But `default_scope` is famously hard to escape, and Sidekiq workers legitimately run with no tenant in `RequestStore` — they would silently see nothing, or need `unscoped` scattered through them.

Chosen: C — an explicit `in_current_tenant` scope via a `TenantScopedThrough` concern, deriving tenancy from `sessions.tenant_id` over an association path. No denormalisation, no backfill, and the requirement stays visible at every call site. Outside a request it returns `all`, matching how `TenantScoped` already behaves, so workers are unaffected.

What it costs: it is opt-in, so a new controller can forget it. That is exactly the risk in Constraint Signal #1, and I would rather have it visible and documented than hidden inside a `default_scope` that someone eventually fights.

Decision 4 — How much to change

I deliberately did **not** touch the live interview WebSocket path, the coverage state engine, or the audio pipeline. They are the largest and most intricate parts of the codebase and they were not where the harm was. The brief asks for the change that makes the product genuinely better, not the one that touches the most lines. One vertical slice — schema through service through API through type through pixel — with tests, is worth more than broad shallow edits.

Step 5 — Monozukuri Execution

What shipped

Eight commits on `feat/fit-gap-integrity`.

Commit	
`chore(dev)`	Docker stack, dev image, demo seed, `.gitattributes`
`feat(db)`	Three migrations making absence representable
`fix(api)`	Generator: coercion, atomicity, override preservation, PII
`fix(api)`	Engine: provenance, unassessed accounting, matching
`fix(api)`	`TenantScopedThrough` and every unscoped lookup
`fix(web)`	Contract repair, unassessed visibility, states
`test`	RSpec + Vitest harnesses
`docs`	Seeded-fault evidence

Migration safety

All three are reversible. The two that relax `NOT NULL` **refuse to roll back** while unrepresentable rows exist, rather than backfilling:

```
raise ActiveRecord::IrreversibleMigration, <<~MSG
  #{unrated} portfolio_skills row(s) have ai_level IS NULL and cannot be
  expressed under the previous NOT NULL schema.
  ...
  Do not backfill them to 1 — that is the defect this migration removed.
MSG
```

Backfilling to `1` would restore the schema and silently re-harm those candidates during what an operator believes is a neutral rollback. Making the operator decide is the safer default.

A detail I verified rather than assumed: in PostgreSQL a `CHECK` evaluates to `UNKNOWN` against `NULL` and the row passes, so `chk_portfolio_skills_ai_level` admits `NULL` unchanged while still rejecting `0` and `6`. No constraint modification was needed.

Verified by running `migrate → rollback → migrate` against a populated database.

Test coverage

Both harnesses built from zero — `.rspec` existed with no `spec/` directory, and the web app had no runner.

```
54 examples, 0 failures        # RSpec
Tests  18 passed (18)          # Vitest
```

The suites deliberately cover the paths that were broken, not the happy path. Notable examples:

- five parametrised cases proving `nil`, `""`, `"three"`, `0`, `9` all yield `nil` rather than `1`
- an unusable response leaves an existing portfolio intact
- a mid-batch failure rolls the entire rewrite back — using a real 300-character label against the `varchar(255)` column rather than a stub, so it exercises the actual failure
- overrides survive regeneration, retaining the overruled `ai_level`
- `generation_error` and the error log contain no transcript content
- cross-tenant reads and writes raise `RecordNotFound` — **with a companion example asserting that the unscoped lookup still finds the record**, so the test documents the old behaviour rather than merely forbidding the new one
- the Required cell is populated, and the summary counts sum to the number of required skills

Seeded fault test

Full evidence in `assessment/evidence/`. Both the fault and its revert are in history:

```
075e96e  spike: seed two faults to prove the suites catch them
49f1636  Revert "spike: seed two faults to prove the suites catch them"
```

Fault 1 — restored `value.to_i.clamp(1, 5)`. **22 examples, 7 failures:**

Model output	Expected	Got
`nil`, `""`, `"three"`, `0`	`nil`	`1`
`9`	`nil`	`5`
`"L4"`	`4`	`1`
confidence when unrated	`low`	`high`

Fault 2 — restored `required_level` on the client. **3 failed, 15 passed.** The third failure is the original defect's fingerprint:

```
skills that were not assessed > shows a dash rather than a level
  → Found multiple elements with the text: –
```

With the fault present the Required cell also falls back to an em dash, so an unassessed row renders *two*. That is precisely what the screenshot of the unfixed application shows.

After revert: 54 and 18, green.

AI verification moments

The brief asks for one. I have two worth recording, and they cut in opposite directions.

Where I nearly shipped a false claim

I read `PortfoliosController#fitgap` and found this comment:

```
# Return cached report if it exists and portfolio has no new overrides
existing = FitGapReport.find_by(portfolio_id: portfolio.id, vacancy_id: vacancy.id)
if existing
  return json_response(report: fit_gap_json(existing))
end
```

The comment promises an override check. The code performs none. I had already drafted this as a P0 — "*an assessor's override never propagates to the fit/gap report*" — because the reasoning was clean and the evidence looked damning.

Before writing it up I went looking for the write path, and found `PortfolioSkillsController#override` calls `regenerate_stale_fitgap_reports`, which destroys affected reports and re-queues generation. **The staleness is handled, from the other side.** The comment is misleading; the behaviour is correct. I deleted the finding.

I have kept this because it is the failure mode I most distrust in AI-assisted analysis: a plausible chain, real supporting evidence, and a conclusion that is simply wrong because the disproof lives in a file the reasoning never opened. A stale comment is exactly the kind of artefact that will confirm a hypothesis you already hold.

(There is a real but narrower issue here — `report.destroy` followed by `perform_async` leaves a window where the client 404s and re-triggers generation, so two workers can race the unique index. That is a P2 I have documented and not fixed.)

Where the type system caught the machine

Changing `PortfolioSkill.ai_level` from `string` to `number | null` immediately produced three compile errors in files I had not looked at — `OverridePanel.tsx` and `SkillPortfolioCard.tsx` both passed it to `parseLevel`, which was typed to reject `null`.

Following those led to P0-2: `parseLevel` had its own `return isNaN(n) ? 1 : n` fallback. I had believed the L1 coercion was a backend defect. It was in both layers, and I would not have found the frontend half without making the type honest first. Making a type accurate is a way of asking the compiler where you were wrong.

Two smaller ones, for completeness: I wrote a test stubbing `save!` via `and_wrap_original` that failed with an arity error — I replaced the stub with a real 300-character label, which is a better test anyway. And I asserted `getByTitle(long)` on a string with a trailing space, not knowing Testing Library normalises whitespace before

comparing. Both were caught by running the tests rather than by reading them.

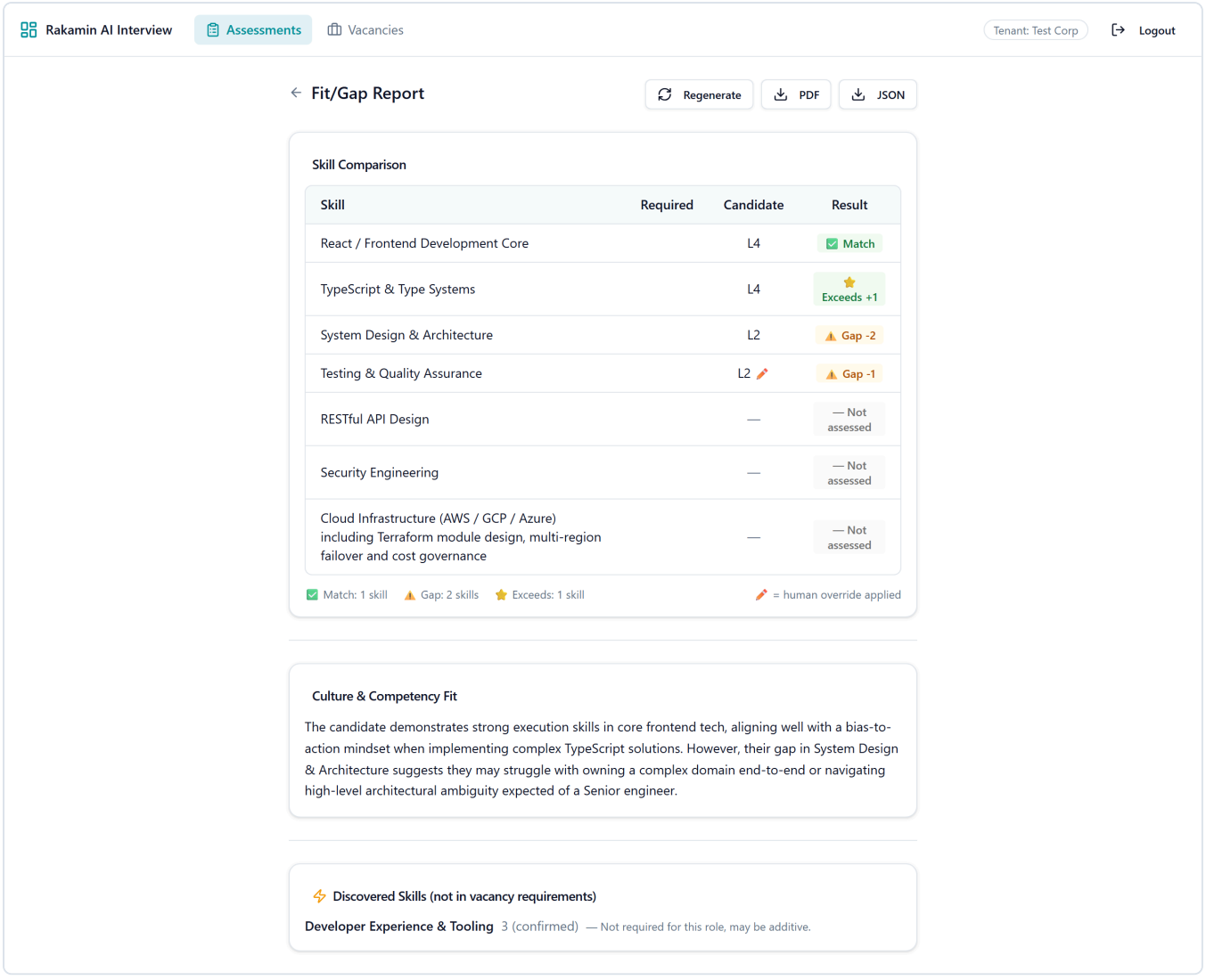
What I did not do

- The live interview WebSocket, coverage state engine and audio pipeline are untouched.
- No CI pipeline. The suites run in one command each and would drop into GitHub Actions in a few lines; I prioritised the shipped slice.
- No prompt evaluation harness — see Constraint Signal #5.
- `GEMINI_LIVE_MODEL` is left as found; the live audio API uses a different model family and I could not verify a working id for it without conducting a live session.
- No request specs for the tenancy fix, only model-level. The model specs prove the scope; a request spec would additionally prove the controllers use it. That is the gap I would close first.

Step 6 — Screenshots

All captured against the running application with the seeded demo candidate — a real Gemini narrative, a real assessor override, and three genuinely unassessed skills. Reproducible via the Playwright script noted at the end of this section.

Before — `main`

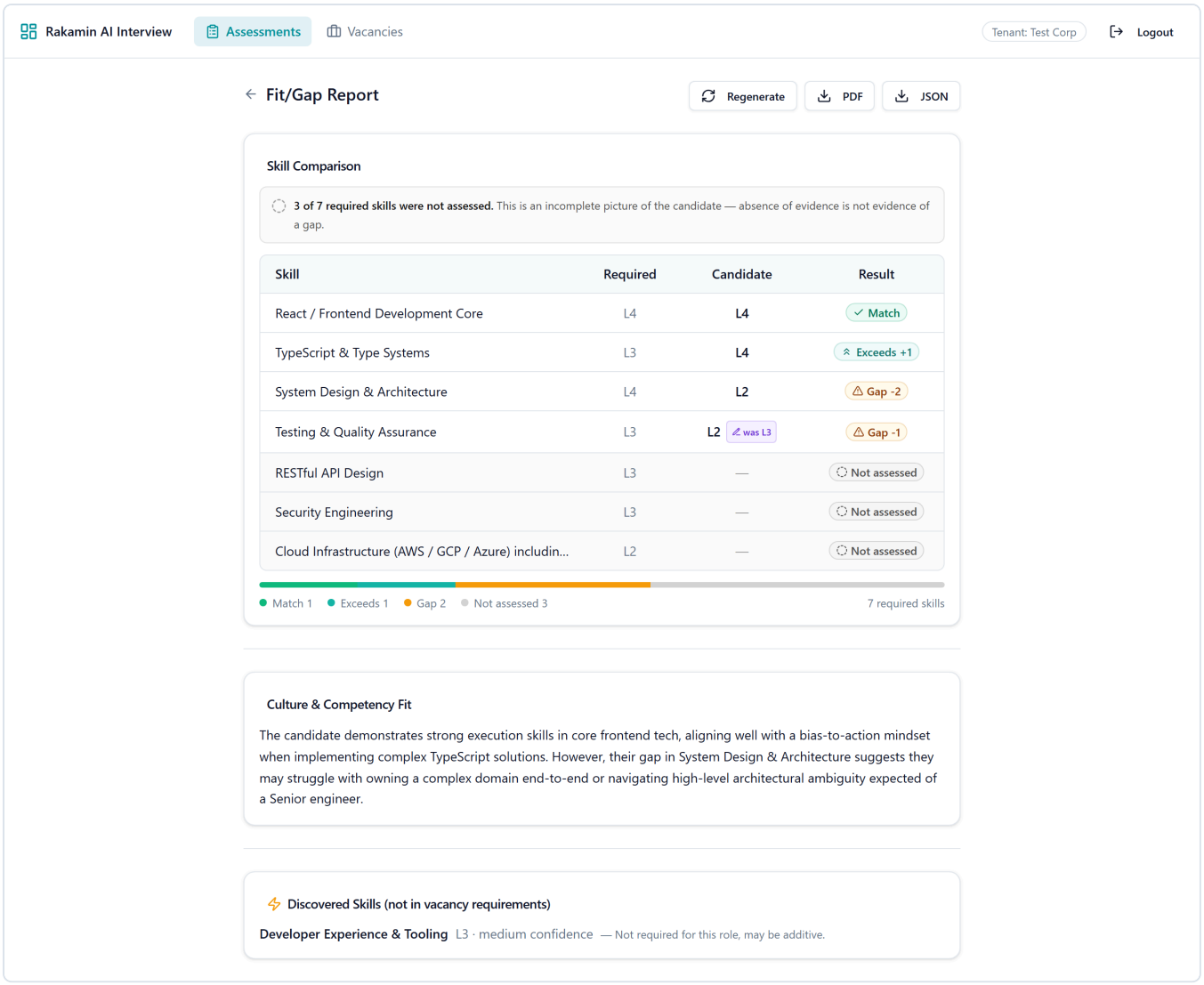


This is `main`'s frontend, served from a `git worktree` against the same API and the same data.

- **The Required column is empty on every row.** A gap report that cannot state what the gap is measured from.
- **The summary reads `Match: 1 · Gap: 2 · Exceeds: 1`** — four of seven required skills. RESTful API Design, Security Engineering and Cloud Infrastructure appear as rows but in no count and no total.
- The long Cloud Infrastructure label runs the full width of the table.

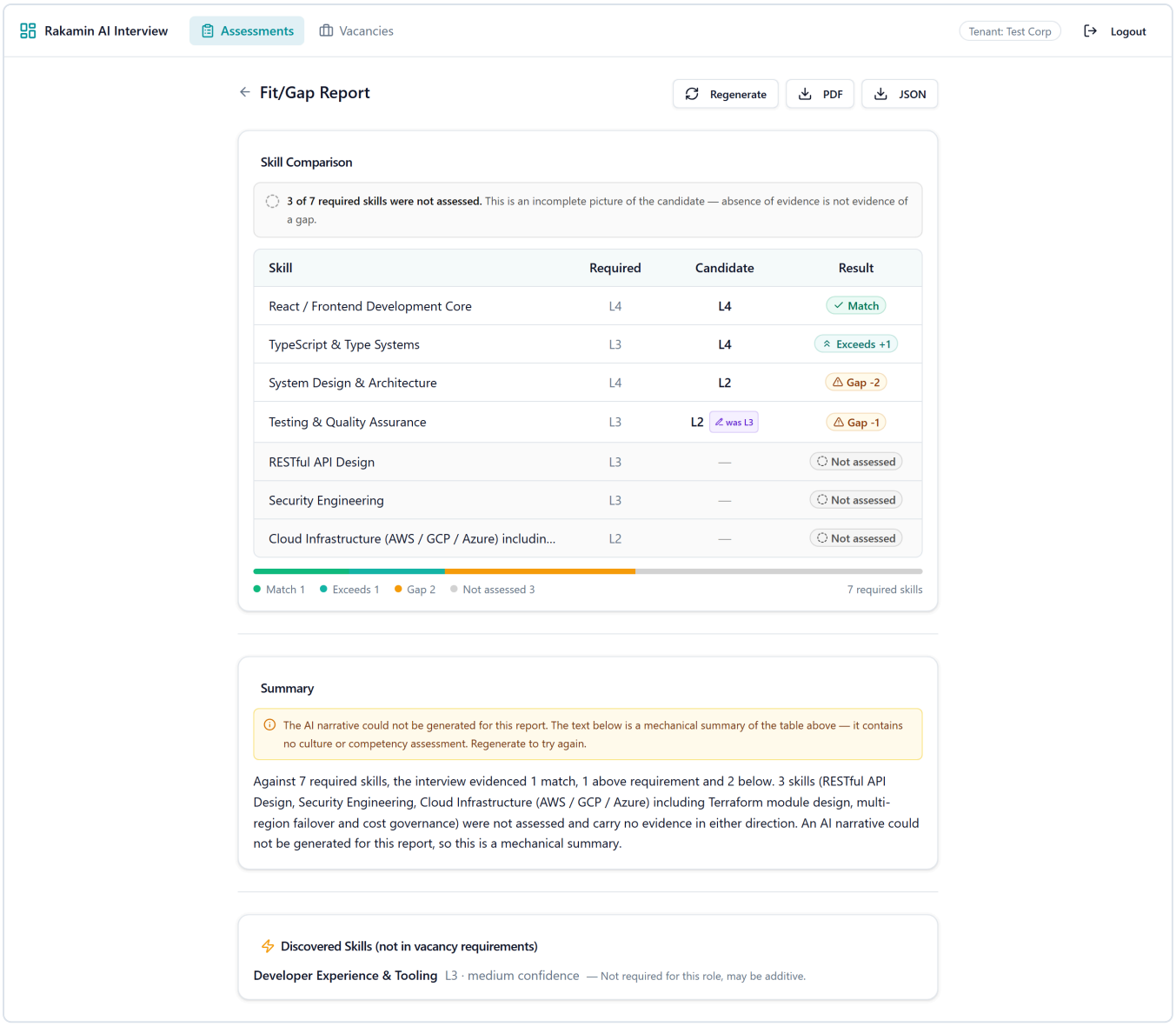
One honest caveat: the ✎ marker beside `L2` on Testing & Quality Assurance **does** render here, because this old client is talking to my fixed API, which now sends `is_override`. Against the original backend that flag never arrived and the marker never appeared — the legend was advertising something impossible. If anything this makes the point more sharply: the client-side code for provenance was always present and had simply been disconnected from a server that never sent the field.

After



- **Required is populated on every row**, unassessed ones included.
- A statement above the table: "*3 of 7 required skills were not assessed. This is an incomplete picture of the candidate — absence of evidence is not evidence of a gap.*"
- `L2 ✎ was L3` — the assessor's level and the model level it replaced, with the full attribution on hover.
- A proportional coverage bar, and a legend that totals **7 required skills**.
- Unassessed rows are neutral grey, never red. They are missing information, not a finding.
- The 255-character label truncates with the full text preserved on hover and for assistive tech.
- Discovered skills read `L3 · medium confidence` — previously `3 (confirmed)`.

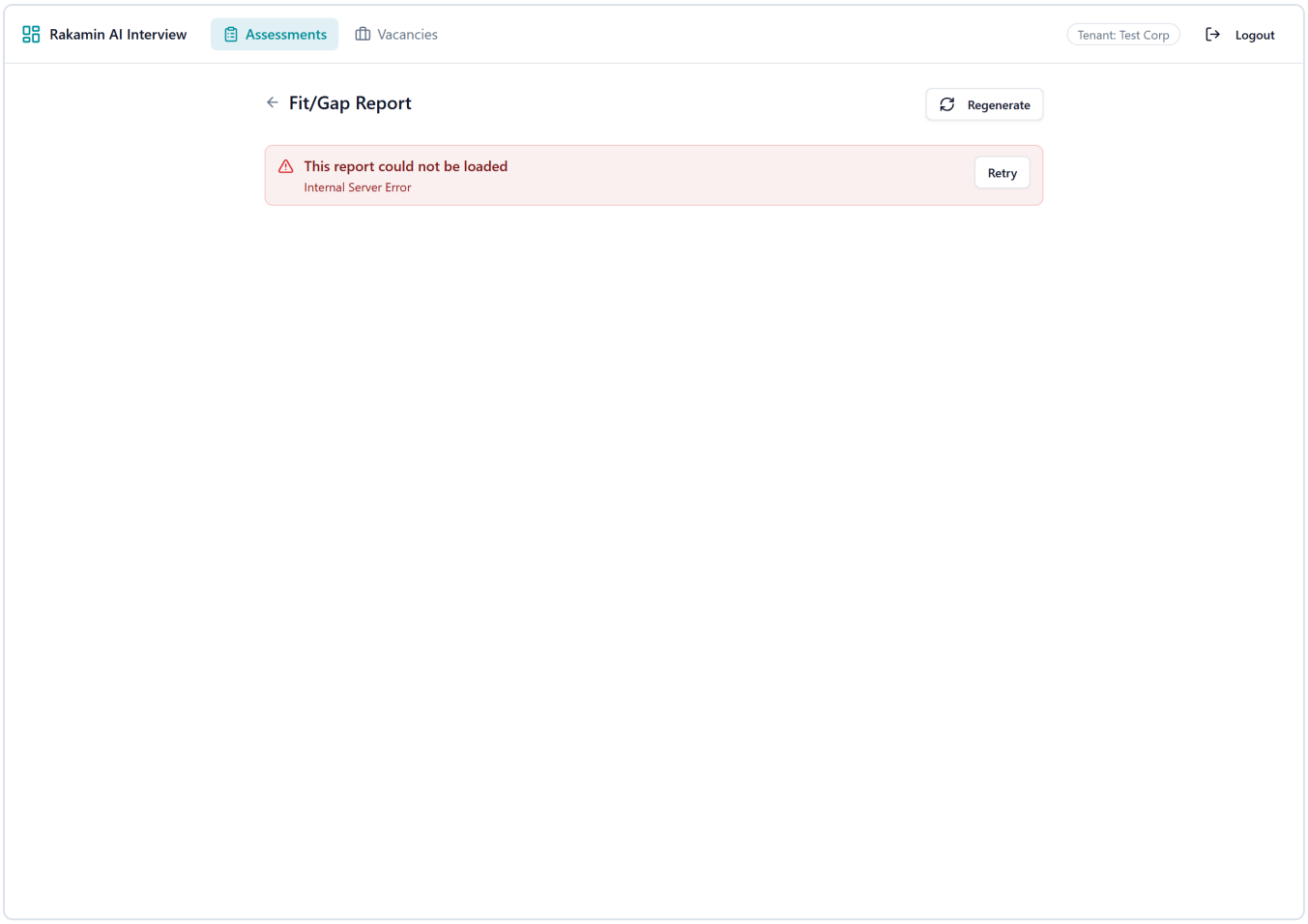
Degraded narrative



When the model call fails, the card retitles itself **Summary**, carries an explicit warning that the text below is mechanical and contains no culture assessment, and the narrative names every unassessed skill. Previously this stub rendered silently under "Culture & Competency Fit".

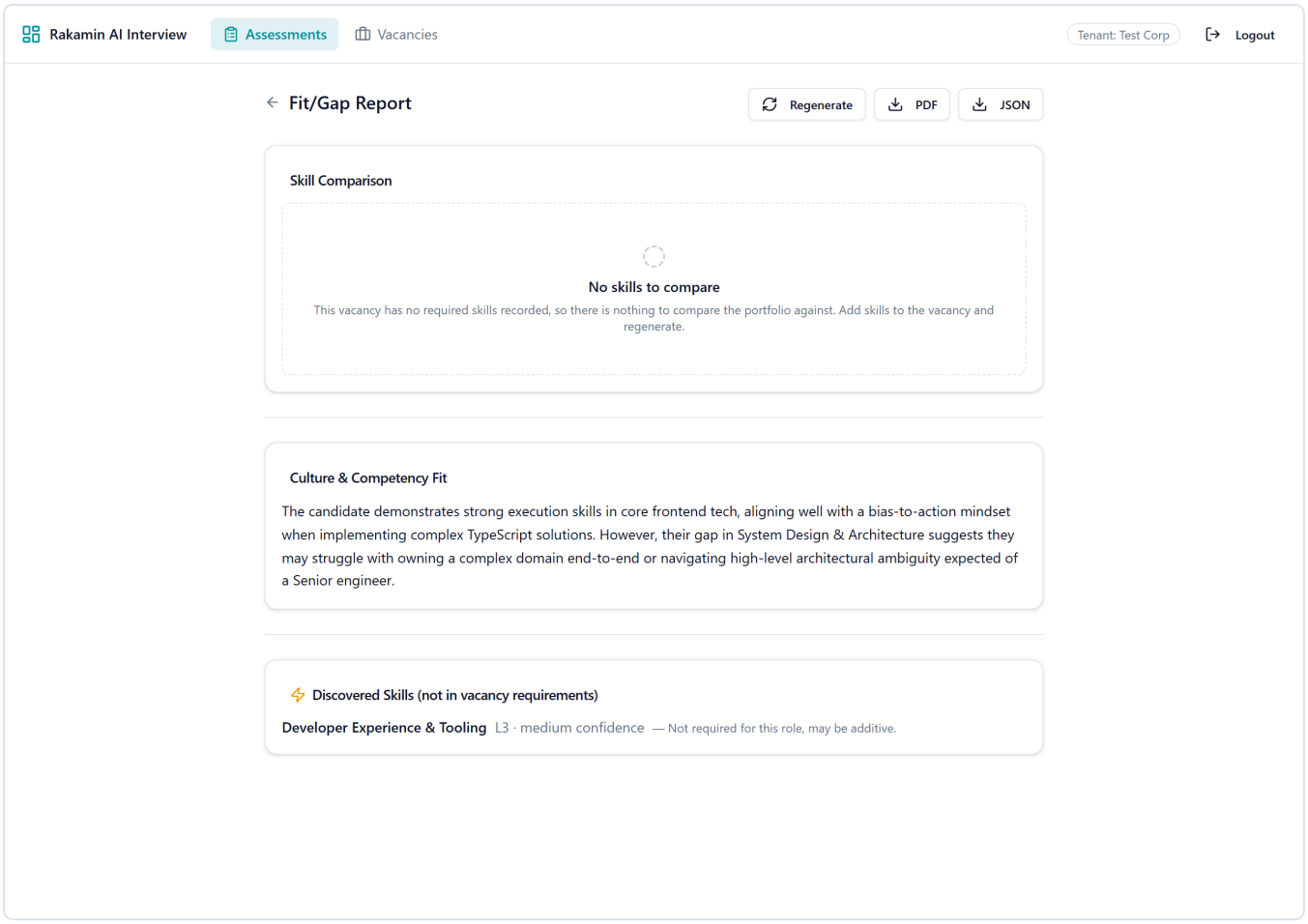
This is also the state I spent an hour in without realising, because the shipped Gemini model ids no longer resolve — which is exactly why `narrative_status` had to exist.

Error



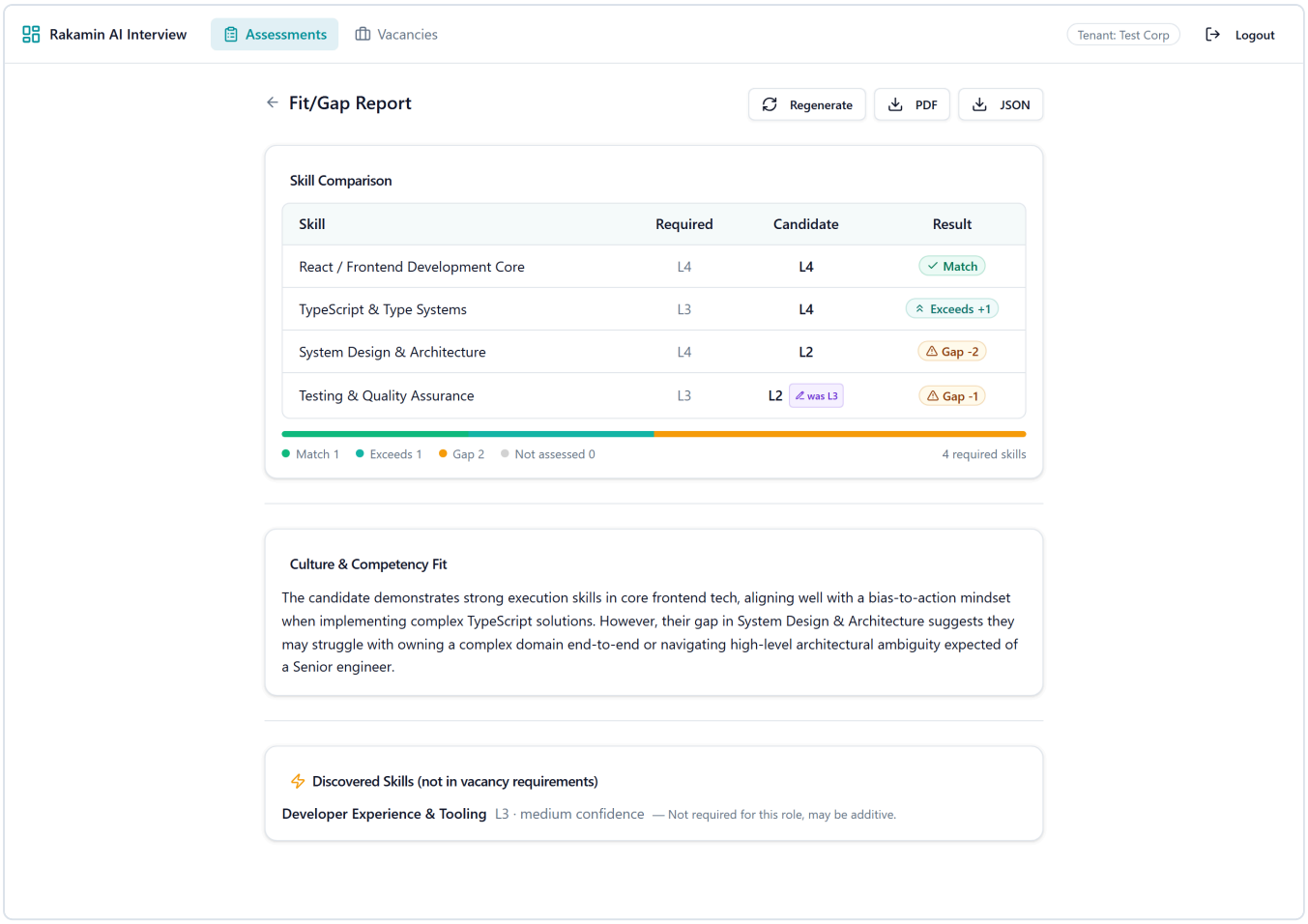
A 500 now produces a titled error, the server's message, and a Retry. Previously every non-404 was swallowed into a blank page.

Empty



A vacancy with no required skills. Previously this raised `RecordInvalid` inside the worker, which retried and never terminated.

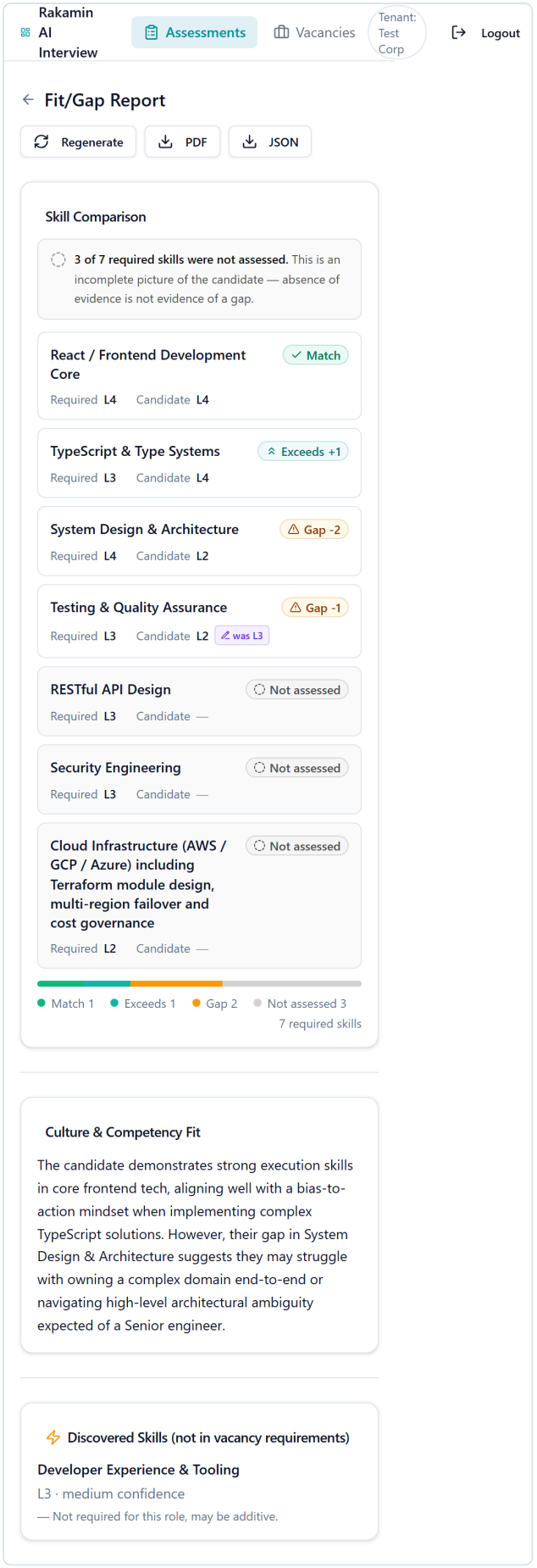
Fully assessed



The incompleteness banner is absent when there is nothing to warn about — the warning has to be informative, not decorative.

Responsive

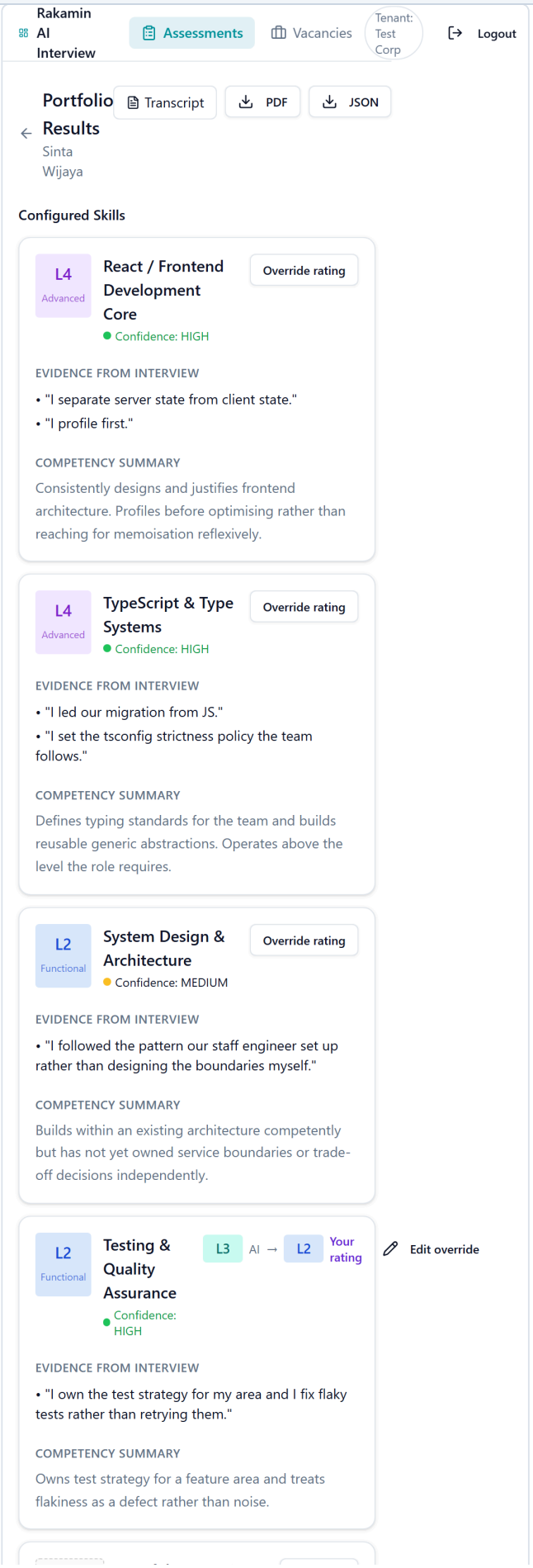
Fit/gap, 390 px	Portfolio, 390 px

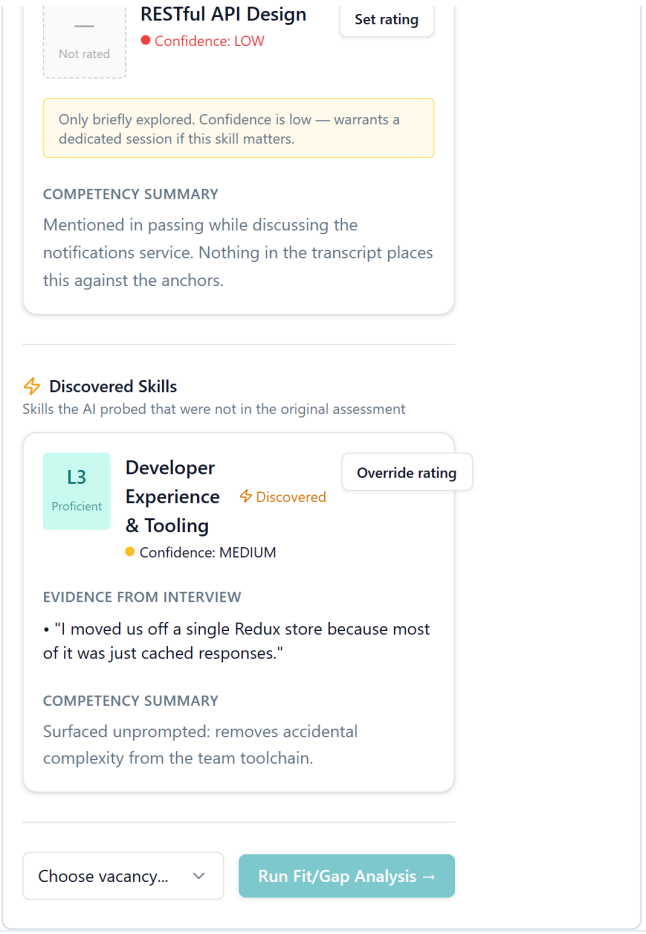


Fit/gap, 390 px	Portfolio, 390 px

Fit/gap, 390 px

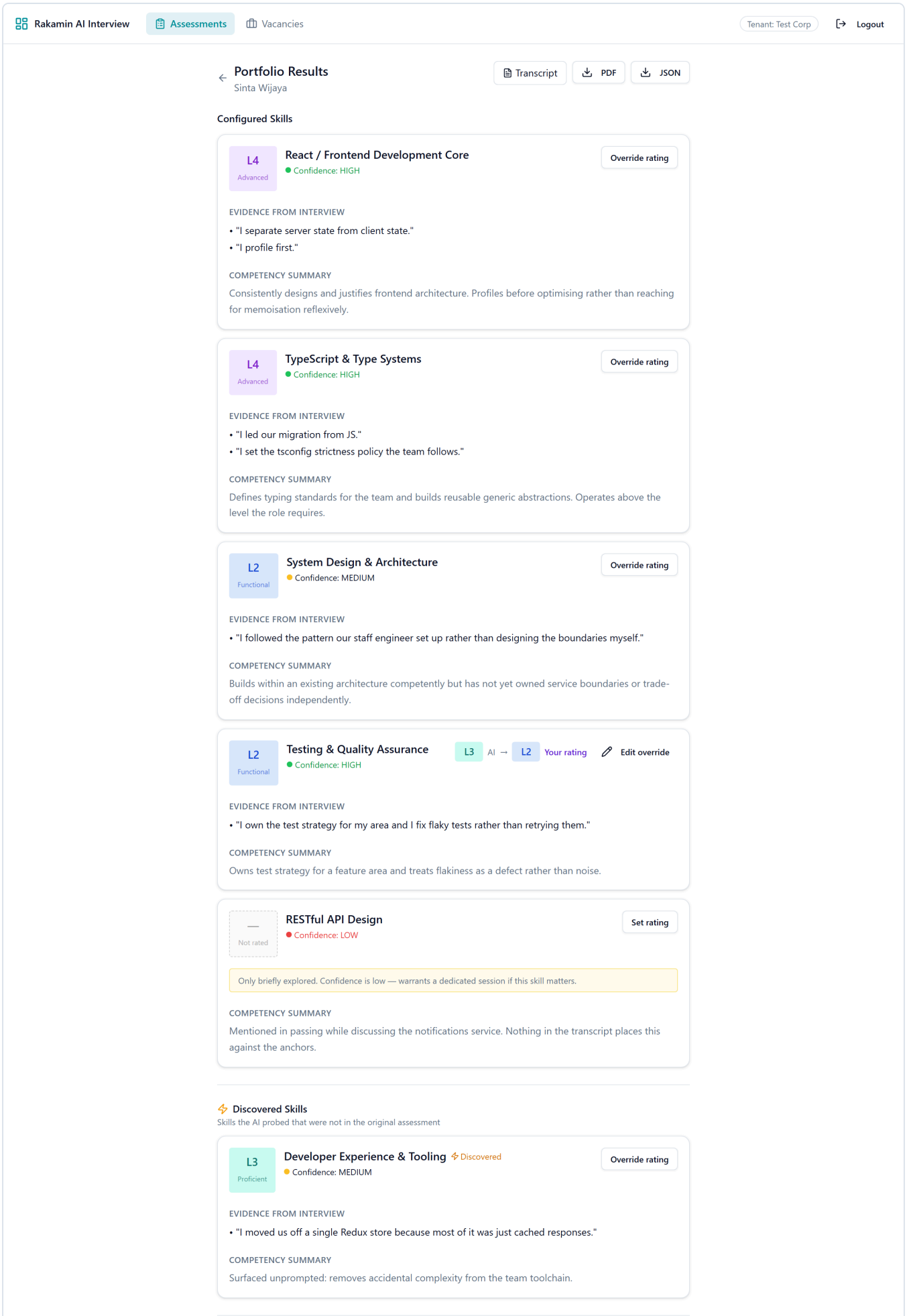
Portfolio, 390 px

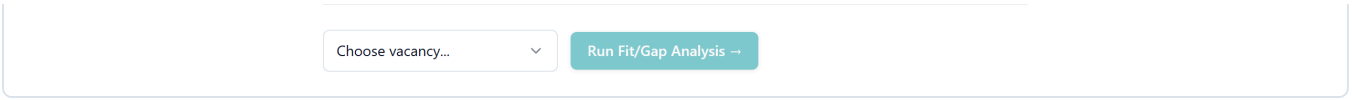




Below `sm` the table is replaced by stacked cards rather than left to scroll horizontally, so Required and Candidate stay side by side at the width they are actually compared at.

Portfolio

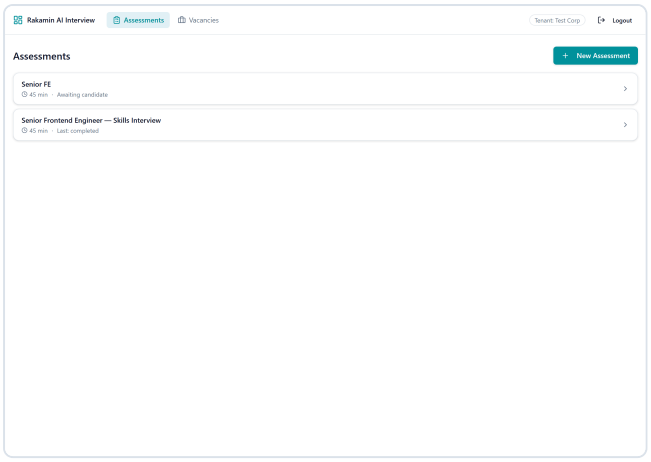

Rakamin AI Interview
Assessments
Vacancies
Tenant: Test Corp
Logout
Portfolio Results
Sinta Wijaya
Transcript
PDF
JSON
Configured Skills
L4
Advanced
React / Frontend Development Core
Confidence: HIGH
Override rating
EVIDENCE FROM INTERVIEW
• "I separate server state from client state."
• "I profile first."
COMPETENCY SUMMARY
Consistently designs and justifies frontend architecture. Profiles before optimising rather than reaching for memoisation reflexively.
L4
Advanced
TypeScript & Type Systems
Confidence: HIGH
Override rating
EVIDENCE FROM INTERVIEW
• "I led our migration from JS."
• "I set the tsconfig strictness policy the team follows."
COMPETENCY SUMMARY
Defines typing standards for the team and builds reusable generic abstractions. Operates above the level the role requires.
L2
Functional
System Design & Architecture
Confidence: MEDIUM
Override rating
EVIDENCE FROM INTERVIEW
• "I followed the pattern our staff engineer set up rather than designing the boundaries myself."
COMPETENCY SUMMARY
Builds within an existing architecture competently but has not yet owned service boundaries or trade-off decisions independently.
L2
Functional
Testing & Quality Assurance
Confidence: HIGH
L3
AI →
L2
Your rating
Edit override
EVIDENCE FROM INTERVIEW
• "I own the test strategy for my area and I fix flaky tests rather than retrying them."
COMPETENCY SUMMARY
Owns test strategy for a feature area and treats flakiness as a defect rather than noise.
—
Not rated
RESTful API Design
Confidence: LOW
Set rating
Only briefly explored. Confidence is low — warrants a dedicated session if this skill matters.
COMPETENCY SUMMARY
Mentioned in passing while discussing the notifications service. Nothing in the transcript places this against the anchors.
Discovered Skills
Skills the AI probed that were not in the original assessment
L3
Proficient
Developer Experience & Tooling
Discovered
Confidence: MEDIUM
Override rating
EVIDENCE FROM INTERVIEW
• "I moved us off a single Redux store because most of it was just cached responses."
COMPETENCY SUMMARY
Surfaced unprompted: removes accidental complexity from the team toolchain.



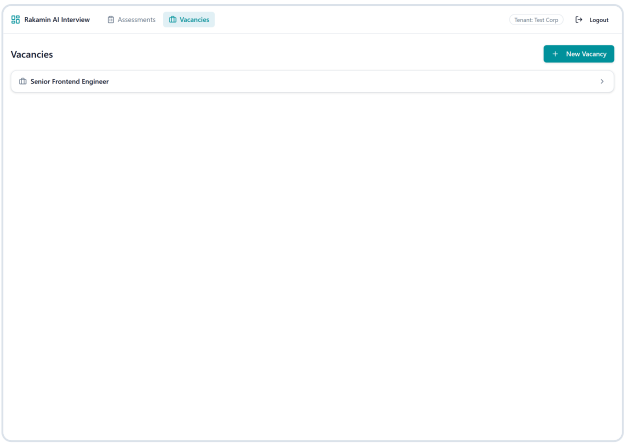
The unrated skill carries a dashed grey badge reading **Not rated** rather than borrowing L1's styling, and its override panel is labelled *Set rating* with a note that anything recorded is the assessor's own judgement, attributed to them. Saving is disabled until a level is chosen — no pre-selected default, because a default is how an unassessed skill acquires a rating nobody intended.

The rest of the product

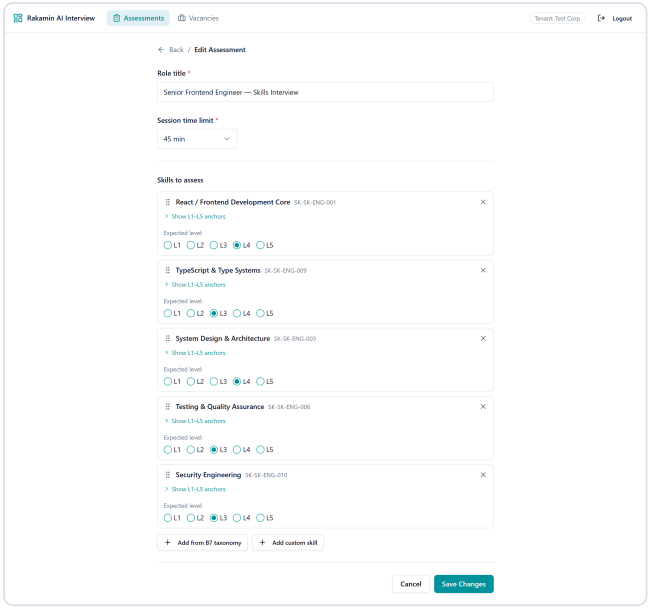
The change above is one vertical slice. For context, these are the surfaces it sits inside — untouched by this PR except where the design system needed the same states.



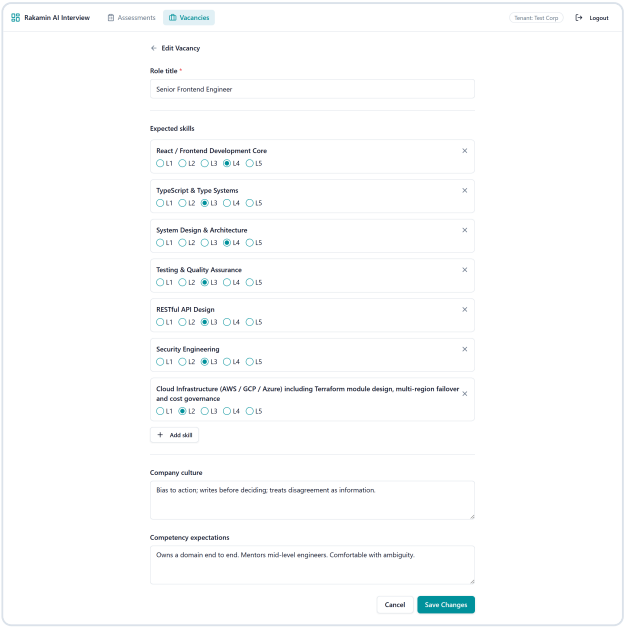
Assessments — where an assessor configures skills and a time limit



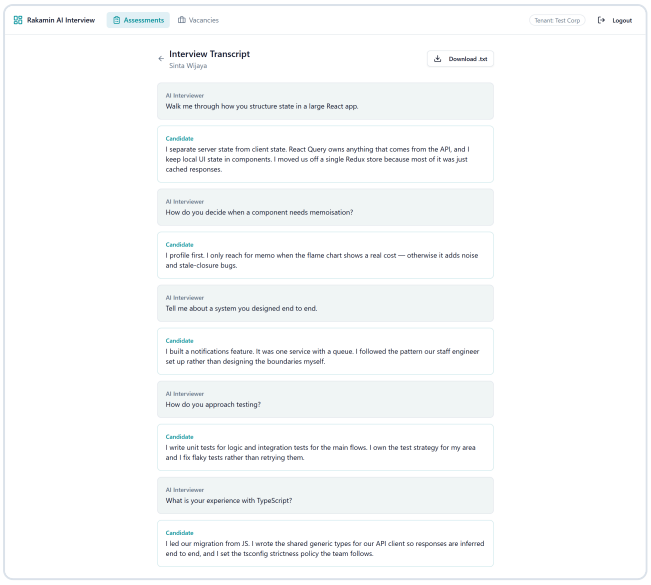
Vacancies — the role definition the portfolio is compared against



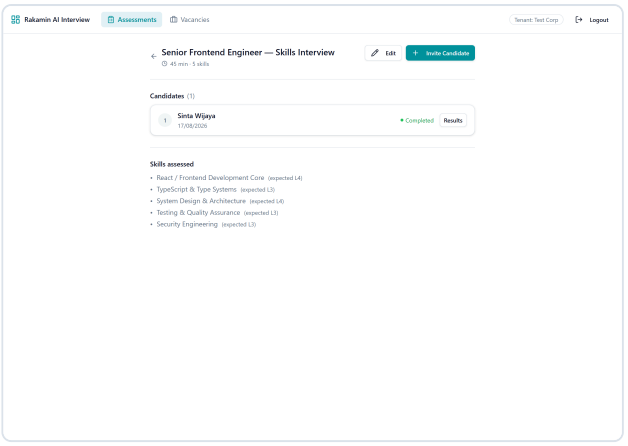
Skill configuration — each skill carries five behavioural anchors from the shared taxonomy



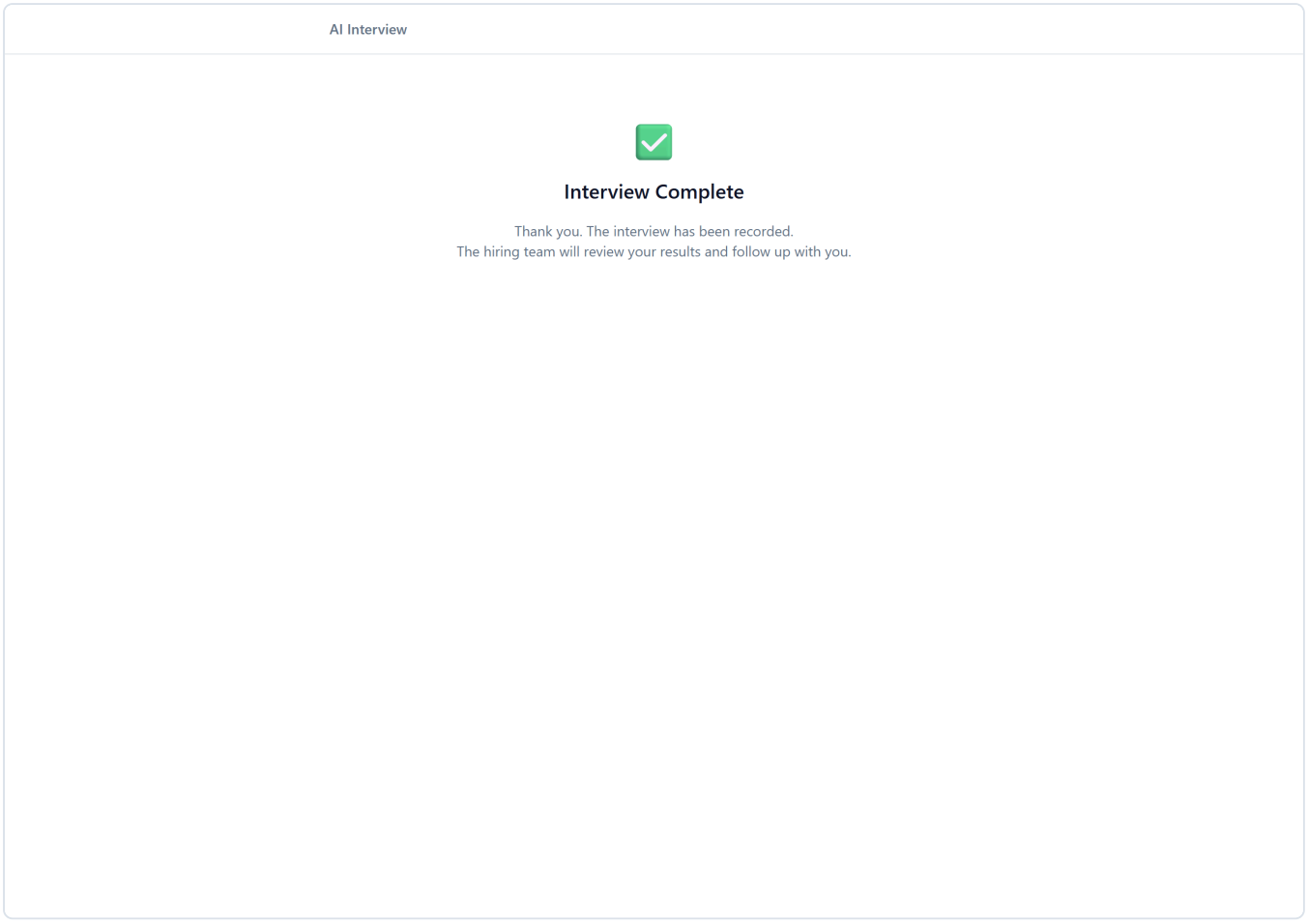
Expected levels — the right-hand side of every fit/gap comparison



Transcript — the evidence every level is supposed to be traceable to



Invite — how a candidate reaches the interview



The candidate's own view. Worth looking at while thinking about UU PDP: this is the only screen the person being assessed ever sees, and it is the last one. They receive no copy of the portfolio, the levels, or the narrative written about them.

Reproducing these

`tooling/capture.mjs` (outside the repo — the PR should not carry a browser binary to produce report images) drives headless Chromium, resolves the seeded ids from the API rather than hardcoding them, and waits on rendered content rather than a timer. The first run of it captured eight skeleton loaders, which is its own small lesson about verifying output rather than exit codes.

Video

https://youtu.be/bs6h74YCspc

A walkthrough of the same argument this report makes, in order: what the report looked like before, the `to_i.clamp(1, 5)` line that caused it, the fixed screen with the unassessed skills stated and counted, the cross-tenant read and write, both suites running, the seeded fault failing, and the two things I would escalate rather than claim as finished.

Appendix — Running it

```
cp .env.example .env          # add your Google AI Studio key
docker compose up -d --build
docker compose exec api bundle exec rails db:create db:migrate db:seed
docker compose exec api bundle exec rails demo:seed
cd web && npm install && npm run dev
```

Tests:

```
docker compose exec -e RAILS_ENV=test api bundle exec rspec
cd web && npm test
```

The demo seed prints the URLs for the portfolio and fit/gap screens.